

SparseGPT — Hessian-Aware One-Shot Pruning, Worked Out by Hand

WHY “PRUNE THE SMALLEST WEIGHT” IS WRONG — AND HOW TO PRUNE AND COMPENSATE INSTEAD

What this document does. It takes one tiny linear layer — two output rows, three input columns — and prunes a single weight *by hand*, two ways. The naive way (magnitude: zero the smallest $|W|$) and the SparseGPT way (zero the lowest *impact* weight, then adjust the survivors to absorb the damage). Every number is produced by a short NumPy script (Appendix A, `m06_t01_sparsegpt_engine.py`) so the arithmetic is exact. By the end you will see, on paper, why SparseGPT’s reconstruction error is $27.8\times$ smaller than naive magnitude pruning on the very same layer.

The one idea. A weight’s importance is *not* its magnitude. It is how much output error appears when you remove it *given that you are allowed to fix up the remaining weights*. That “given” is the whole game: it is encoded in the inverse Hessian $H^{-1} = (X^T X)^{-1}$ of the calibration data.

CONCEPTS COVERED, IN ORDER

the reconstruction-loss view of a layer; the input Hessian $H = X^T X$; the SparseGPT impact score $W_{ij}^2/[H^{-1}]_{jj}$; why it differs from magnitude; the OBS compensation update; the exact predicted loss increase; and a side-by-side numeric comparison.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The layer under study	2
2	Step 1 — The reconstruction-loss view	2
3	Step 2 — The Hessian and its inverse	3
4	Step 3 — The SparseGPT impact score	3
5	Step 4 — The OBS compensation update	4
6	Step 5 — The payoff, measured	4
7	Step 6 — From one weight to a whole layer	5
8	The argument, at a glance	6
8.1	Equation cheat-sheet	6
	Appendix A — Reference implementation	6

1 The layer under study

A real pruning pass runs on weight matrices of shape $[d \times d]$ with $d = 4096$ or larger, one matrix at a time, calibrated on a few hundred sample activations. We fix one toy instance so every quantity is checkable.

Quantity	Symbol	Value here
Weight matrix	W	$\begin{pmatrix} 0.6 & -1.0 & 2.0 \\ 1.0 & 3.0 & -2.0 \end{pmatrix}$ $[2 \times 3]$
Calibration set	X	5 input vectors, 3 features each
Target sparsity	—	prune 1 weight (illustrative)

The layer computes $y = Wx$. We have $N = 5$ calibration inputs stacked as rows of $X \in \mathbb{R}^{5 \times 3}$:

$$X = \begin{pmatrix} 3 & 0 & 1 \\ 3 & 1 & 0 \\ 3 & 1 & 1 \\ 2 & 0 & 1 \\ 3 & 1 & 0 \end{pmatrix}.$$

Notice column 0 is driven *hard* (values around 3) while columns 1–2 are small. That asymmetry is deliberate; it is exactly what magnitude pruning is blind to.

Intuition

Pruning a weight is not free even if the weight is small. If the *input* that the weight multiplies is large and frequent, then a small weight there still contributes a meaningful, consistent term to the output — and removing it shifts every output. Conversely, a large weight on an input that is usually near zero barely matters. “Importance” has to fold in the data, not just the weight.

2 Step 1 — The reconstruction-loss view

SparseGPT does not ask “does pruning change the final token?” That is too global. It asks, layer by layer: *after pruning, how different is this layer’s output on the calibration data?* That is a clean least-squares objective:

Per-layer reconstruction loss

$$\mathcal{L}(\widehat{W}) = \sum_{n=1}^N \|\widehat{W}x_n - Wx_n\|_2^2 = \|X\widehat{W}^\top - XW^\top\|_F^2.$$

Here W is the original (teacher) weight and $\widehat{W}$ the pruned-and-repaired weight. The loss is zero when $\widehat{W} = W$ and grows as we damage it. The whole of SparseGPT is: *choose which entries of $\widehat{W}$ to force to zero, and choose the remaining entries, to keep $\mathcal{L}$ as small as possible.*

Why the Hessian is $X^\top X$

$\mathcal{L}$ is quadratic in $\widehat{W}$. For one output row $w \in \mathbb{R}^3$ the loss is $\|X(\widehat{w} - w)\|^2$, whose Hessian with respect to $\widehat{w}$ is

$$\frac{\partial^2 \mathcal{L}}{\partial \widehat{w}^2} = 2 X^\top X \equiv 2H, \quad H = X^\top X.$$

A quadratic with known Hessian is exactly the setting where Optimal Brain Surgeon (OBS) gives a *closed-form* answer for “zero this entry and optimally repair the rest.” No iteration, no gradient descent. This is what makes one-shot pruning possible.

3 Step 2 — The Hessian and its inverse

Compute $H = X^\top X$ from the five calibration rows:

$$H = X^\top X = \begin{pmatrix} 40 & 9 & 8 \\ 9 & 3 & 1 \\ 8 & 1 & 3 \end{pmatrix}, \quad H^{-1} = \begin{pmatrix} 0.2759 & -0.6552 & -0.5172 \\ -0.6552 & 1.9310 & 1.1034 \\ -0.5172 & 1.1034 & 1.3448 \end{pmatrix}.$$

The diagonal of the inverse is what we need:

$$[H^{-1}]_{00} = 0.2759, \quad [H^{-1}]_{11} = 1.9310, \quad [H^{-1}]_{22} = 1.3448.$$

Read the inverse-Hessian diagonal as “ease of repair”

$[H^{-1}]_{jj}$ is *small* for column 0 (0.28) because that input is large and frequent — the other weights have little freedom to compensate for changes there, so $[H^{-1}]_{jj}$ being small will *amplify* a weight’s impact. $[H^{-1}]_{jj}$ is *large* for column 1 (1.93) — that direction is poorly constrained by the data, so damage there is easy to absorb. The inverse Hessian encodes “how hard is it to fix this column after a change.”

4 Step 3 — The SparseGPT impact score

Now the central formula. The cost of zeroing weight W_{ij} and *optimally repairing the rest of its row* is, from OBS:

SparseGPT impact score / optimal removal cost

$$\text{impact}(i, j) = \frac{W_{ij}^2}{[H^{-1}]_{jj}}.$$

This is exactly the increase in $\mathcal{L}$ caused by setting $W_{ij} = 0$ once the surviving weights have absorbed what they can.

Plug in. Dividing each squared weight by the matching diagonal entry:

$$\text{impact} = \begin{pmatrix} \frac{0.6^2}{0.2759} & \frac{(-1)^2}{1.9310} & \frac{2^2}{1.3448} \\ \frac{1^2}{0.2759} & \frac{3^2}{1.9310} & \frac{(-2)^2}{1.3448} \end{pmatrix} = \begin{pmatrix} 1.305 & 0.518 & 2.974 \\ 3.625 & 4.661 & 2.974 \end{pmatrix}.$$

The lowest-impact weight is $W_{0,1} = -1.0$ with impact 0.518 — *that* is what SparseGPT prunes.

Watch out

Magnitude and impact disagree here. The smallest-magnitude weight is $W_{0,0} = 0.6$. Magnitude pruning would zero it. But $W_{0,0}$ sits on the high-variance column 0 (small $[H^{-1}]_{00}$), so its impact is 1.305 — *higher* than $W_{0,1}$'s impact of 0.518, even though $|0.6| < |-1.0|$. Magnitude picks the wrong weight to remove. This single disagreement is why the two methods will diverge by more than an order of magnitude below.

5 Step 4 — The OBS compensation update

Choosing *which* weight to zero is only half of SparseGPT. The other half is *repairing the rest of the row* so the output barely moves. Suppose we zero entry (i, q) . OBS gives the optimal adjustment to every other weight in that row:

OBS optimal compensation for zeroing entry (i, q)

$$\delta w_{ij} = -\frac{W_{iq}}{[H^{-1}]_{qq}} [H^{-1}]_{qj} \quad \text{for } j \neq q, \quad \Delta \mathcal{L} = \frac{W_{iq}^2}{[H^{-1}]_{qq}}.$$

We prune $W_{0,1} = -1.0$, so $q = 1$, $[H^{-1}]_{11} = 1.9310$, and the scale factor is $-W_{0,1}/[H^{-1}]_{11} = -(-1)/1.9310 = 0.5179$. Multiply by the row $[H^{-1}]_{1,:}$:

Compute the compensation δ for row 0

$$\begin{aligned} \delta w_{0,0} &= 0.5179 \times [H^{-1}]_{1,0} = 0.5179 \times (-0.6552) = -0.3393, \\ \delta w_{0,1} &= 0 \quad (\text{this is the pruned column itself}), \\ \delta w_{0,2} &= 0.5179 \times [H^{-1}]_{1,2} = 0.5179 \times (1.1034) = +0.5714. \end{aligned}$$

So row 0 changes from $(0.6, -1.0, 2.0)$ to

$$\hat{w}_0 = (0.6 - 0.3393, \ 0, \ 2.0 + 0.5714) = (0.2607, \ 0, \ 2.5714). \quad \checkmark$$

The survivors $W_{0,0}$ and $W_{0,2}$ *shifted* to soak up the loss of $W_{0,1}$. They lean on the correlations between input columns (the off-diagonal H^{-1} entries) to keep Wx almost unchanged on the calibration set.

Intuition

Think of the three input features as three teammates who together produce the right answer. When you bench feature 1's contribution (zero its weight), the other two players nudge their own effort up or down — by amounts the inverse Hessian dictates — to cover the gap. Naive magnitude pruning benches a player and tells nobody; the score just drops.

6 Step 5 — The payoff, measured

Now evaluate the reconstruction loss $\mathcal{L}$ for each strategy on the calibration set.

Strategy	Weight pruned	Repair	$\mathcal{L}$
Naive magnitude	$W_{0,0} = 0.6$ (smallest $ W $)	none	14.400
SparseGPT — zero only	$W_{0,1} = -1.0$ (lowest impact)	none	3.000
SparseGPT — zero + OBS	$W_{0,1} = -1.0$	compensate row	0.518

Two separate wins stack up:

1. **Better choice.** Even with *no* repair, pruning the lowest-impact weight ($\mathcal{L} = 3.0$) beats pruning the smallest-magnitude one ($\mathcal{L} = 14.4$). Magnitude pruned a weight on a high-energy input and paid dearly.
2. **Repair.** Adding the OBS compensation drops the loss from 3.0 to 0.518 — and that 0.518 *exactly* matches the predicted $\Delta\mathcal{L} = W_{0,1}^2/[H^{-1}]_{11} = 1/1.9310 = 0.518$. The closed form is not an approximation here; it is the realized loss. ✓

The number to remember

On this layer, SparseGPT’s reconstruction error is $14.400/0.518 = \mathbf{27.8\times}$ smaller than naive magnitude pruning, for removing the *same number of weights*. Most of that gap is the smarter choice of *which* weight; the rest is the compensation. At scale ($d = 4096$, 50% sparsity, hundreds of columns), these per-weight wins compound into the multi-point MMLU gap between magnitude RTN and SparseGPT.

7 Step 6 — From one weight to a whole layer

The real algorithm repeats this column by column. Processing column j :

1. score every entry in the column by $W_{ij}^2/[H^{-1}]_{jj}$;
2. zero the lowest-scoring fraction (e.g. 50%, or exactly 2-of-4 for a 2:4 mask — see Topic 2);
3. push the OBS compensation *forward* onto the not-yet-processed columns, using the off-diagonal H^{-1} entries, so later columns see the error and absorb it.

Because the inverse Hessian is computed once (via a Cholesky factorization) and reused for every column, the cost is a few minutes per layer and *no retraining*. That “one-shot, no gradient” property is what made SparseGPT (and its lighter cousin Wanda) the default for 50%-sparse LLMs.

Watch out

The damping term: in practice one adds a small ridge, $H \leftarrow H + \lambda \text{diag}(\bar{H})$ with $\lambda \approx 0.01$, before inverting. Real activation Hessians are near-singular (many correlated features), and without damping H^{-1} blows up and the compensation over-corrects. Our toy H is well-conditioned so we skipped it; never skip it on a real layer.

8 The argument, at a glance

#	Step	Result
1	Loss view	minimize $\ X\widehat{W}^\top - XW^\top\ _F^2$
2	Hessian	$H = X^\top X$; need $\text{diag}(H^{-1})$
3	Score	$\text{impact} = W_{ij}^2/[H^{-1}]_{jj}$; prune $W_{0,1}$
4	Repair	$\delta w_0 = (-0.339, 0, +0.571)$
5	Payoff	$\mathcal{L}$: 14.4 $\rightarrow$ 0.518, i.e. 27.8 $\times$ ✓
6	Scale	column-by-column, one-shot, no retraining

8.1 Equation cheat-sheet

Reconstruction loss	$\mathcal{L} = \ X\widehat{W}^\top - XW^\top\ _F^2$
Input Hessian	$H = X^\top X$
Impact / removal cost	$\text{impact}(i, j) = W_{ij}^2/[H^{-1}]_{jj}$
OBS compensation	$\delta w_{ij} = -(W_{iq}/[H^{-1}]_{qq}) [H^{-1}]_{qj}$
Realized loss rise	$\Delta\mathcal{L} = W_{iq}^2/[H^{-1}]_{qq}$
Magnitude (contrast)	score = $ W_{ij} $ (ignores H)

Appendix A — Reference implementation

Every number above is produced by `m06_t01_sparsegpt_engine.py`. Run it to reproduce all figures; change W or X to see how the impact ranking and the compensation move.

```

1  """
2  Reference implementation for: Module 06 - Topic 1
3  "SparseGPT - Hessian-aware one-shot pruning, prune + compensate."
4
5  A deliberately tiny example (2 output rows, 3 input columns) so every number
6  in the worked-by-hand PDF can be checked by hand. We prune ONE weight (the
7  lowest SparseGPT impact score) and run the OBS compensation update on the
8  surviving weights, then compare reconstruction error against naive magnitude
9  pruning of the SAME weight count.
10
11  Layer model: y = W x, with W [out=2, in=3], calibrated on N input vectors X.
12  Hessian of the squared reconstruction loss over the calibration set is
13  H = X^T X (in x in). SparseGPT impact(i,j) = W[i,j]^2 / [H^-1]_jj.
14  """
15
16  import numpy as np
17  np.set_printoptions(precision=4, suppress=True)
18
19  # -----
20  # 0. A tiny linear layer + its calibration data.
21  # -----
22  W = np.array([[ 0.6, -1.0,  2.0],
23               [ 1.0,  3.0, -2.0]])      # [out=2, in=3]
24  # Calibration activations: N=5 input vectors (rows), in=3 columns.
25  # Column 0 is driven hard (large variance) so its H^-1_00 is tiny; a small
26  # weight there is actually HARD to remove -> magnitude and impact disagree.
27  X = np.array([[3.0, 0.0, 1.0],
28               [3.0, 1.0, 0.0],
29               [3.0, 1.0, 1.0],
30               [2.0, 0.0, 1.0],
31               [3.0, 1.0, 0.0]])      # [N=5, in=3]

```

```

32 out, in_ = W.shape
33 N = X.shape[0]
34 print("== 0. Setup ==")
35 print("W =\n", W)
36 print("X (calibration, N=5) =\n", X)
37
38 # -----
39 # 1. Input Hessian H = X^T X and its inverse.
40 # -----
41 H = X.T @ X
42 Hinv = np.linalg.inv(H)
43 print("\n== 1. Hessian H = X^T X ==")
44 print("H =\n", H)
45 print("H^-1 =\n", Hinv)
46 print("diag(H^-1) =", np.diag(Hinv))
47
48 # -----
49 # 2. SparseGPT impact score impact(i,j) = W[i,j]^2 / [H^-1]_jj.
50 # -----
51 hinv_diag = np.diag(Hinv)
52 impact = (W**2) / hinv_diag[None, :]
53 print("\n== 2. SparseGPT impact scores W^2 / [H^-1]_jj ==")
54 print("impact =\n", impact)
55 flat_min = np.unravel_index(np.argmin(impact), impact.shape)
56 print(f"lowest-impact weight = W[{flat_min[0]},{flat_min[1]}] = {W[flat_min]:.4f}"
57       f" (impact={impact[flat_min]:.4f})")
58
59 # For contrast: pure magnitude would prune the smallest |W|.
60 mag = np.abs(W)
61 mag_min = np.unravel_index(np.argmin(mag), mag.shape)
62 print(f"magnitude would prune W[{mag_min[0]},{mag_min[1]}] = {W[mag_min]:.4f}"
63       f" (|W|={mag[mag_min]:.4f})")
64
65 # -----
66 # 3. Reconstruction error baseline: how well W reproduces the teacher
67 # output on the calibration set. We measure the change in output
68 # caused by pruning, in the SAME row as the pruned weight.
69 # For a single pruned entry (row i, col q), the OBS-optimal update to
70 # the REMAINING weights in that row is:
71 # delta w_j = - (w_q / [H^-1]_qq) * [H^-1]_qj for j != q
72 # and the resulting increase in the layer loss is
73 # dL = w_q^2 / [H^-1]_qq (= the impact score).
74 # -----
75 def layer_loss(Wmat):
76     """Sum of squared error of (Wmat x) vs (W x) over calibration set."""
77     diff = X @ (Wmat - W).T # [N, out]
78     return np.sum(diff**2)
79
80 i, q = flat_min # prune this entry
81 print(f"\n== 3. Prune W[{i},{q}] two ways, compare reconstruction loss ==")
82
83 # --- (a) naive: just zero it, do nothing else ---
84 W_naive = W.copy()
85 W_naive[i, q] = 0.0
86 L_naive = layer_loss(W_naive)
87 print(f"(a) naive zero, no compensation : loss = {L_naive:.5f}")
88
89 # --- (b) SparseGPT OBS: zero it AND compensate the rest of row i ---
90 W_obs = W.copy()
91 wq = W[i, q]
92 # delta on the remaining columns of this row
93 delta = - (wq / Hinv[q, q]) * Hinv[q, :]
94 delta[q] = 0.0 # column q itself is zeroed, not updated

```

```

95 W_obs[i, :] = W[i, :] + delta
96 W_obs[i, q] = 0.0
97 L_obs = layer_loss(W_obs)
98 print(f"(b) SparseGPT OBS compensation : loss = {L_obs:.5f}")
99 print(f"    compensation delta to row {i} = {delta}")
100 print(f"    new row {i} = {W_obs[i]}")
101 print(f"predicted dL = w_q^2/[H^-1]_qq = {wq**2 / Hinv[q,q]:.5f} (matches OBS loss)")
102 print(f"\nimprovement: OBS loss is {L_naive/L_obs:.2f}x lower than naive on this weight")
103
104 # -----
105 # 4. Now contrast pruning CHOICE: SparseGPT vs magnitude pick different
106 #    weights. Compare each method's best achievable loss (its own pick,
107 #    compensated where the method allows).
108 # -----
109 def obs_prune_one(Wmat, i, q):
110     Wn = Wmat.copy()
111     d = - (Wmat[i, q] / Hinv[q, q]) * Hinv[q, :]
112     d[q] = 0.0
113     Wn[i, :] = Wmat[i, :] + d
114     Wn[i, q] = 0.0
115     return Wn
116
117 print("\n== 4. Full method comparison ==")
118 # Naive magnitude: pick smallest |W|, just zero it (no compensation).
119 W_mag = W.copy(); W_mag[mag_min] = 0.0
120 L_mag_naive = layer_loss(W_mag)
121 # SparseGPT: pick lowest impact, zero it, OBS-compensate the rest of the row.
122 L_sg = layer_loss(obs_prune_one(W, *flat_min))
123 print(f"naive magnitude : prune W[{mag_min[0]},{mag_min[1]}] (|W|={W[mag_min]:.2f}),"
124       f" no comp -> loss = {L_mag_naive:.5f}")
125 print(f"SparseGPT      : prune W[{flat_min[0]},{flat_min[1]}] (impact={impact[flat_min]:.3f}),"
126       f" OBS comp -> loss = {L_sg:.5f}")
127 print(f"SparseGPT loss is {L_mag_naive/L_sg:.2f}x lower than naive magnitude")
128
129 print("\n== summary ==")
130 print(f"naive magnitude (wrong pick, no comp): {L_mag_naive:.5f}")
131 print(f"SparseGPT (impact pick + OBS comp)    : {L_sg:.5f}")
132 print(f"reduction factor                        : {L_mag_naive/L_sg:.2f}x")

```

— END OF DOCUMENT —